Lecture 7: Asynchronous programming

Presentations students, introduction to async

Willem Vanhulle

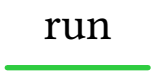
DevLab Rust 2025

Tuesday December 9, 2025

github.com/wvhulle/rust-course-ghent

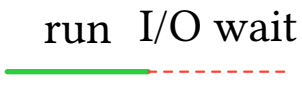
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.



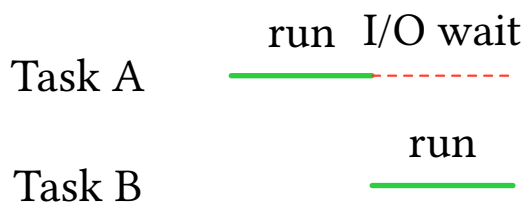
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.



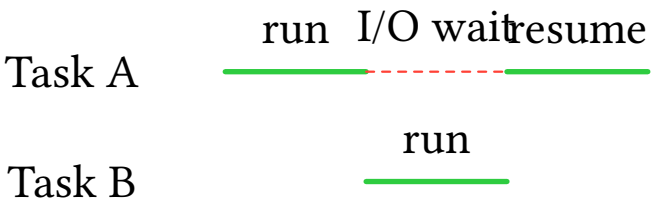
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.



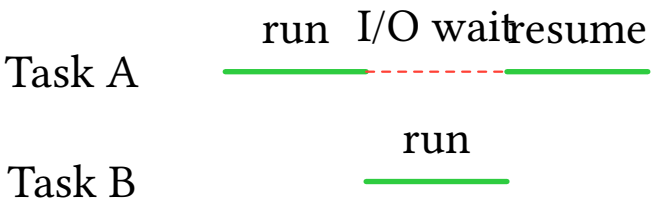
2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

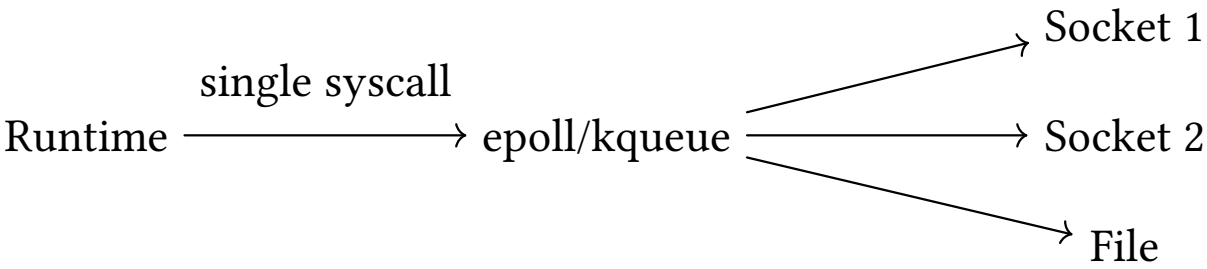


2.1. Introduction

Multiple tasks execute concurrently by running until they would block on I/O, then yielding to another ready task.

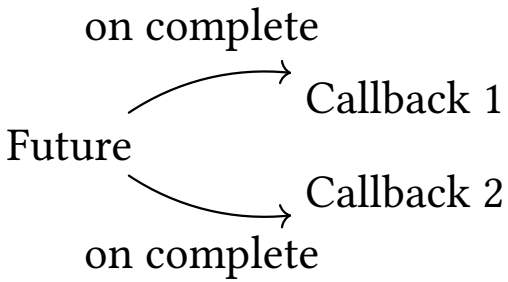


Per-task overhead is minimal because operating systems provide primitives (epoll, kqueue, IOCP) that monitor many I/O operations with a single system call.



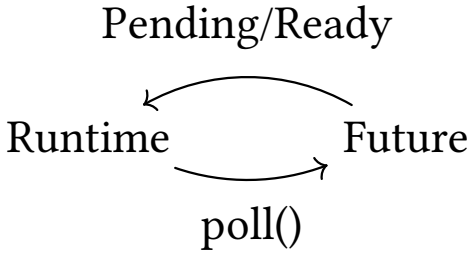
2.2. Comparison with Python

Python asyncio



- Callback-based model
- Register callbacks
- Built-in event loop
- Interpreter overhead

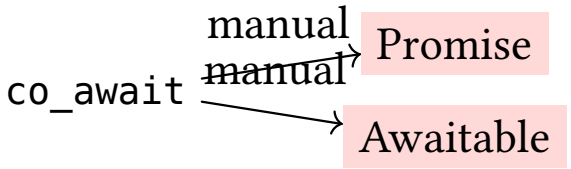
Rust async/await



- Poll-based model
- Runtime polls futures
- External runtimes (tokio, async-std)
- Zero-cost state machines

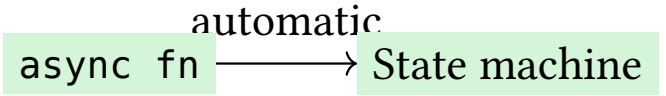
2.3. Comparison with C++

C++20 Coroutines



- Manual promise types
- Manual awaitable objects
- No standard runtime
- Heap allocation by default

Rust async/await



- Compiler generates state machines
- Built-in syntax
- Mature ecosystem (tokio, async-std)
- Stack-allocated by default

3.1. Await syntax

At a high level, async Rust code looks very much like “normal” sequential code:

```rust
use futures::executor::block_on;

async fn count_to(count: i32) {
    for i in 0..count {
        println!("Count is: {i}!");
    }
}

async fn async_main(count: i32) {
    count_to(count).await;
}

fn main() {
    block_on(async_main(10));
}
```

(playground link)

3.2. Futures

Future is a trait, implemented by objects that represent an operation that may not be complete yet. A future can be polled, and poll returns a `Poll`.

```
1   use std::pin::Pin; use std::task::Context;
2
3   pub trait Future {
4       type Output;
5       fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
6   }
7
8   pub enum Poll<T> {
9       Ready(T),
10      Pending,
11  }
```

(playground link)

What is the role of `Context`?

3.2. Futures

Future is a trait, implemented by objects that represent an operation that may not be complete yet. A future can be polled, and poll returns a `Poll`.

```
1   use std::pin::Pin; use std::task::Context;
2
3   pub trait Future {
4       type Output;
5       fn poll(self: Pin<&mut Self>, cx: &mut Context<'_>) -> Poll<Self::Output>;
6   }
7
8   pub enum Poll<T> {
9       Ready(T),
10      Pending,
11  }
```

(playground link)

What is the role of `Context`?

Context allows a Future to schedule itself to be polled again when an event such as a timeout occurs.

3.3. JoinHandle

One common type implementing `Future` is a Tokio `JoinHandle`:

```rust
use tokio::task;

let join = task::spawn(async {
    panic!("something bad happened!")
});

// The returned result indicates that the task failed.
assert!(join.await.is_err());
```

(playground link)

4.1. State machine

Rust transforms async functions into state machines that implement `Future`.

4.1. State machine

Rust transforms async functions into state machines that implement `Future`.

Key characteristics:

- Calling an async function constructs and returns a future
- Does not execute immediately (lazy)
- Tracks progress through suspension points

4.1. State machine

Rust transforms async functions into state machines that implement `Future`.

Key characteristics:

- Calling an async function constructs and returns a future
- Does not execute immediately (lazy)
- Tracks progress through suspension points

The state machines are allocated on the stack by default. When the state machine transitions to the next state, the async runtime does not move it.

4.2. Transformation

Consider this async function:

```
1 async fn two_d10(modifier: u32) -> u32 {
2     let first_roll = roll_d10().await;
3     let second_roll = roll_d10().await;
4     first_roll + second_roll + modifier
5 }
```

(playground link)

4.2. Transformation

Consider this async function:

```
1 async fn two_d10(modifier: u32) -> u32 {
2     let first_roll = roll_d10().await;
3     let second_roll = roll_d10().await;
4     first_roll + second_roll + modifier
5 }
```

(playground link)

The compiler generates an enum that:

- Tracks each suspension point (each `.await`)
- Stores all live variables needed at that state
- Implements the `Future` trait

4.3. State machine structure

The generated enum looks conceptually like this:

```
1 enum TwoDiceFuture {
2     Start { modifier: u32 },
3     FirstRoll { modifier: u32, first_roll: impl Future<Output = u32> },
4     SecondRoll { modifier: u32, first_result: u32, second_roll: impl Future<Output =
      u32> },
5     Done,
6 }
```

(playground link)

4.3. State machine structure

The generated enum looks conceptually like this:

```
1 enum TwoDiceFuture {
2     Start { modifier: u32 },
3     FirstRoll { modifier: u32, first_roll: impl Future<Output = u32> },
4     SecondRoll { modifier: u32, first_result: u32, second_roll: impl Future<Output =
      u32> },
5     Done,
6 }
```

(playground link)

Each state stores:

- The variables still needed
- The future being awaited (if any)

4.4. Recursion

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

4.4. Recursion

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

Recursive async functions require boxing to avoid infinite type sizes:

```
async fn count_to(n: u32) {
    if n > 0 {
        Box::pin(count_to(n - 1)).await;
        println!("{n}");
    }
}

Box::pin(count_to(n - 1)).await;
```

(playground link)

4.4. Recursion

Deeply nested async functions create large compiler-generated Future types.

Each function's Future contains its callees' Futures.

Recursive async functions require boxing to avoid infinite type sizes:

```
1 async fn count_to(n: u32) {
2     if n > 0 {
3         Box::pin(count_to(n - 1)).await;
4         println!("{n}");
5     }
6 }
7
8 Box::pin(count_to(n - 1)).await;                    (playground link)
```

This adds heap allocation overhead but makes recursion possible.

5.1. Runtimes

A runtime provides support for:

- performing operations asynchronously (a **reactor**)
- and is responsible for executing futures (an **executor**).

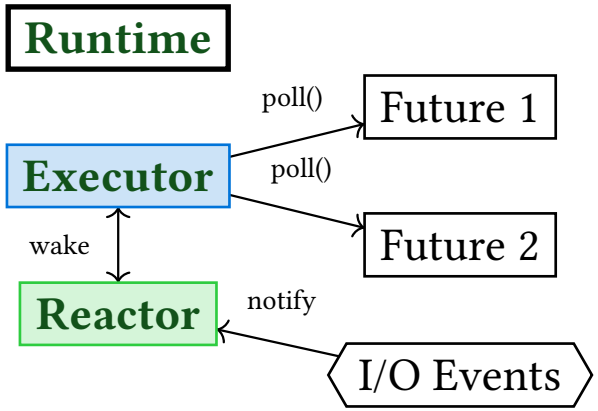
5.1. Runtimes

A runtime provides support for:

- performing operations asynchronously (a **reactor**)
- and is responsible for executing futures (an **executor**).

Rust does not have a "built-in" runtime, but several options are available:

- `futures` offers a bare bones executor `ThreadPool`
- `tokio` offers a `Runtime` with reactor included (supplies "time" etc.)



```rust
use tokio::time;
async fn count_to(count: i32) {
    for i in 0..count {
        println!("Count in task: {i}!");
        time::sleep(time::Duration::from_millis(5)).await;
    }
}
#[tokio::main]
async fn main() {
    tokio::spawn(count_to(10));
    for i in 0..5 {
        println!("Main task: {i}");
        time::sleep(time::Duration::from_millis(5)).await;
    }
}
```

(playground link)

Async tasks are also aborted if not awaited in main.

6.1. Tasks

A task is a top-level future that may be sent to different worker threads (if necessary).

```rust
#[tokio::main]
async fn main() -> io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:0").await?;

    loop {
        let (mut socket, addr) = listener.accept().await?;

        tokio::spawn(async move {
            // Handle connection
        });
    }
}
```

(playground link)

6.1. Tasks

A task is a top-level future that may be sent to different worker threads (if necessary).

```rust
#[tokio::main]
async fn main() -> io::Result<()> {
    let listener = TcpListener::bind("127.0.0.1:0").await?;

    loop {
        let (mut socket, addr) = listener.accept().await?;

        tokio::spawn(async move {
            // Handle connection
        });
    }
}
```

(playground link)

Warning

Although we use `tokio` in this section, the same functionality is available in most async runtimes.

6.1. Tasks

Inside `tokio::spawn`:

```rust
socket.write_all(b"Who are you?\n").await?;

let mut buf = vec![0; 1024];
let name_size = socket.read(&mut buf).await?;
let name = std::str::from_utf8(&buf[..name_size])?.trim();

socket.write_all(format!("Thanks, {name}!\n").as_bytes()).await?;
```

(playground link)

Info

An async block like `async move {}` is like a scope block that may contain awaits.

Refactor `session7/examples/tasks.rs`:

- Create async helper function from async block
- Improve error handling

6.3. Channels

```rust
async fn ping_handler(mut input: mpsc::Receiver<()>) {
    let mut count: usize = 0;
    while let Some(_) = input.recv().await {
        count += 1;
        println!("Received {count} pings so far.");
    }
}
```

(playground link)

```rust
async fn ping_handler(mut input: mpsc::Receiver<()>) {
    let mut count: usize = 0;
    while let Some(_) = input.recv().await {
        count += 1;
        println!("Received {count} pings so far.");
    }
}
```

(playground link)

```rust
async fn main() {
    let (sender, receiver) = mpsc::channel(32);
    let handler_task = tokio::spawn(ping_handler(receiver));
    for i in 0..10 {
        sender.send(()).await.expect("Failed to send");
    }
    drop(sender);
    handler_task.await.expect("Handler failed");
}
```

(playground link)

6.4. Joining

```rust
async fn size_of_page(url: &str) -> Result<usize> {
    let resp = reqwest::get(url).await?;
    Ok(resp.text().await?.len())
}
```

(playground link)

6.4. Joining

```rust
async fn size_of_page(url: &str) -> Result<usize> {
    let resp = reqwest::get(url).await?;
    Ok(resp.text().await?.len())
}
```

(playground link)

```rust
async fn main() {
    let urls = ["https://google.com", "https://httpbin.org/ip", /*...*/];

    let futures = urls.into_iter().map(size_of_page);
    let results = future::join_all(futures).await;

    println!("{results:?}");
}
```

(playground link)

6.5. Select

Waits until any of a set of futures is ready. Similar to `Promise.race` or Python's `asyncio.wait()`.

```rust
#[tokio::main]
async fn main() {
    let (tx, mut rx) = mpsc::channel(32);

    tokio::spawn(async move {
        tokio::select! {
            Some(msg) = rx.recv() => println!("got: {msg}"),
            _ = sleep(Duration::from_millis(50)) => println!("timeout"),
        };
    });

    sleep(Duration::from_millis(10)).await;
    tx.send(String::from("Hello!")).await?;
    // ...
}
```

(playground link)

6.6. Streams

Streams are runtime-agnostic asynchronous iterators:

```rust
use futures::stream::{self, StreamExt};

let stream = stream::iter(vec!['a', 'b', 'c']);

let mut stream = stream.enumerate();

assert_eq!(stream.next().await, Some((0, 'a')));
assert_eq!(stream.next().await, Some((1, 'b')));
assert_eq!(stream.next().await, Some((2, 'c')));
assert_eq!(stream.next().await, None);
```

(playground link)

See my project https://github.com/wvhulle/clone-stream

7.1. Blocking executor

CPU blocking tasks will block the executor and prevent other tasks from being executed.

```rust
async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
    std::thread::sleep(std::time::Duration::from_millis(duration_ms));
    println!("future {id} finished after {}ms", start.elapsed().as_millis());
}

#[tokio::main(flavor = "current_thread")]
async fn main() {
    let start = Instant::now();
    let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
    join_all(futures).await;
}
```

(playground link)

7.1. Blocking executor

CPU blocking tasks will block the executor and prevent other tasks from being executed.

```rust
async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
    std::thread::sleep(std::time::Duration::from_millis(duration_ms));
    println!("future {id} finished after {}ms", start.elapsed().as_millis());
}

#[tokio::main(flavor = "current_thread")]
async fn main() {
    let start = Instant::now();
    let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
    join_all(futures).await;
}
```

(playground link)

What is a workaround?

7.1. Blocking executor

CPU blocking tasks will block the executor and prevent other tasks from being executed.

```rust
async fn sleep_ms(start: &Instant, id: u64, duration_ms: u64) {
    std::thread::sleep(std::time::Duration::from_millis(duration_ms));
    println!("future {id} finished after {}ms", start.elapsed().as_millis());
}

#[tokio::main(flavor = "current_thread")]
async fn main() {
    let start = Instant::now();
    let futures = (1..=10).map(|t| sleep_ms(&start, t, t * 10));
    join_all(futures).await;
}
```

(playground link)

What is a workaround?

Use async methods: `tokio::time::sleep` instead of `std::thread::sleep`

7.2. When to use parallelism

Aspect	Threads (Parallelism)	Async (Concurrency)
Use case	CPU-bound tasks: heavy computation	I/O-bound tasks: waiting for external resources
Goal	Utilize multiple CPU cores for true parallelism	Handle many operations on single thread
Examples	Data processing, scientific computing, compilation, rendering	Web servers, database queries, network clients, file I/O
Overhead	Higher: context switching, separate stacks	Lower: cooperative yielding, shared stack
When best	Independent computations that can run simultaneously	Many concurrent operations that mostly wait

7.3. Deadlocks: Coffman Conditions

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

7.3. Deadlocks: Coffman Conditions

A deadlock occurs when four conditions are met simultaneously (Coffman conditions):

Condition	Description
Mutual exclusion	Resources cannot be shared (only one thread at a time)
Hold and wait	Threads hold resources while waiting for others
No preemption	Resources cannot be forcibly taken from threads
Circular wait	Threads form a cycle waiting for each other's resources

To prevent deadlock, break at least one condition:

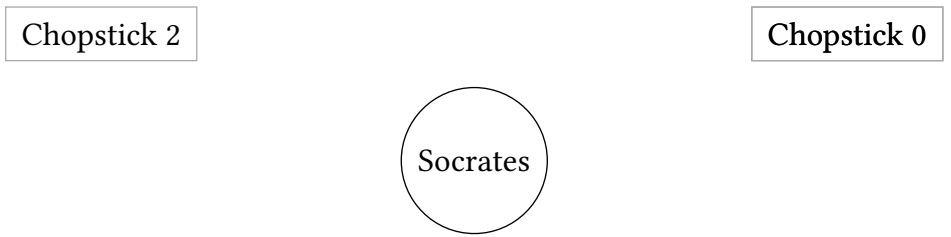
- **Lock ordering**: Always acquire locks in the same order (breaks circular wait)
- **Timeout**: Release locks if waiting too long (breaks hold and wait)
- **Try-lock**: Use `try_lock()` instead of `lock()` (breaks hold and wait)

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

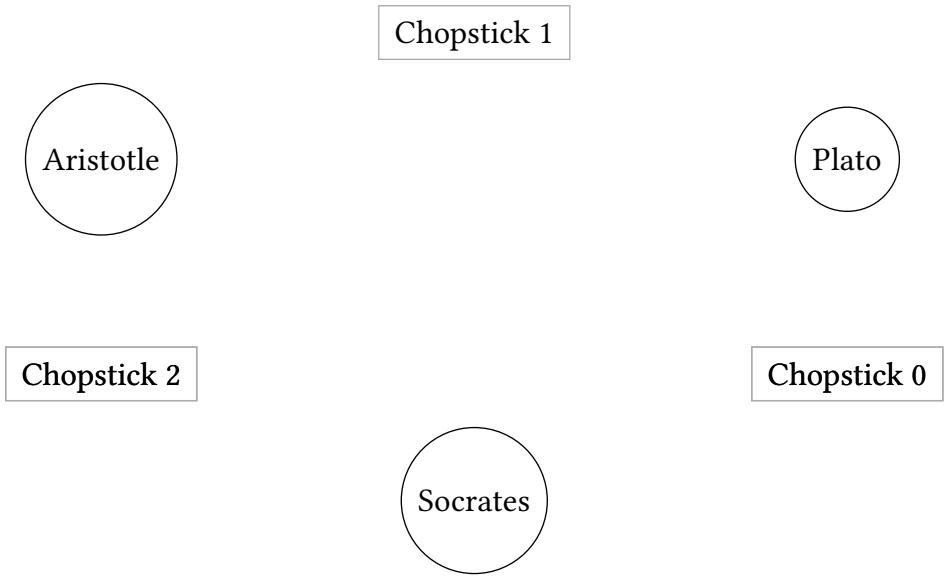


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

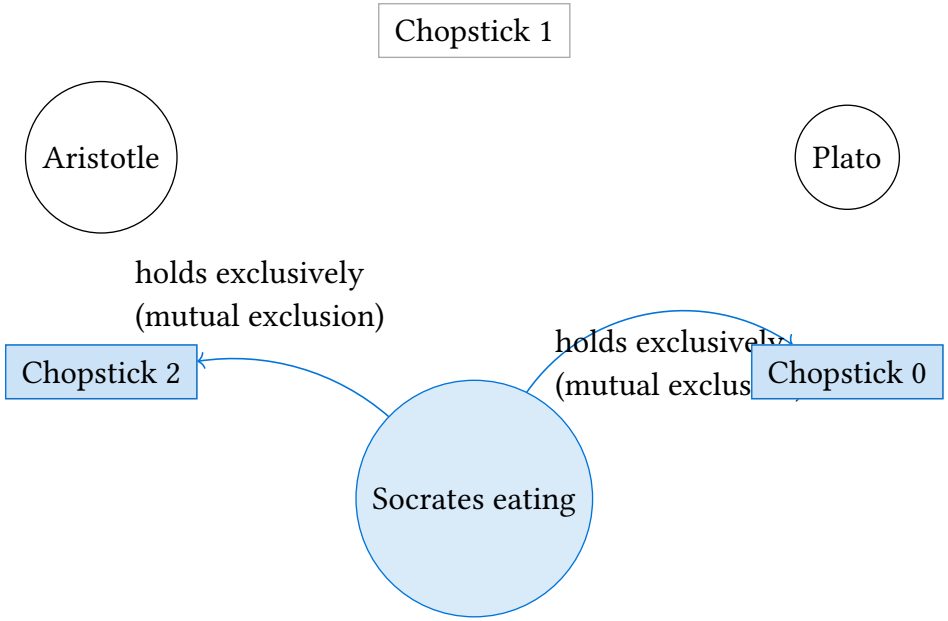


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

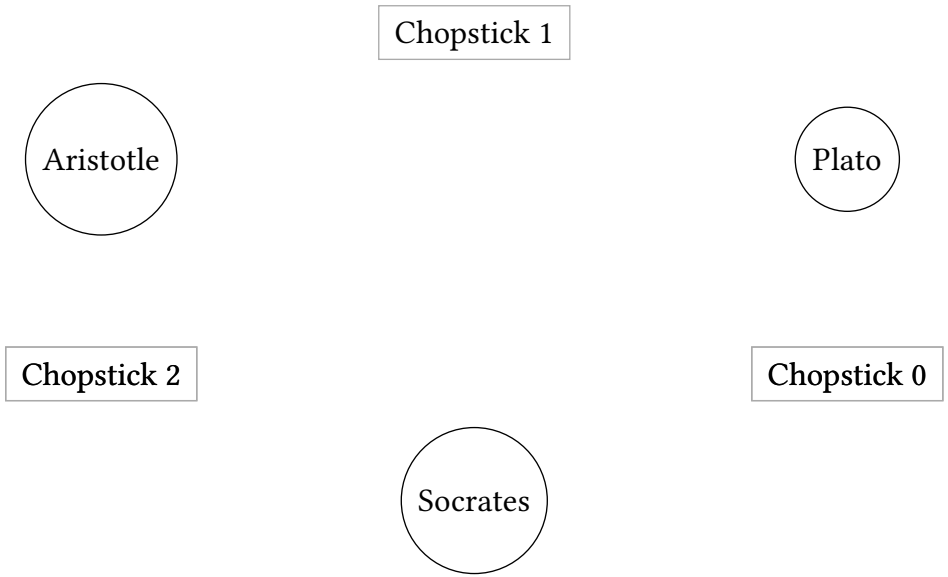


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

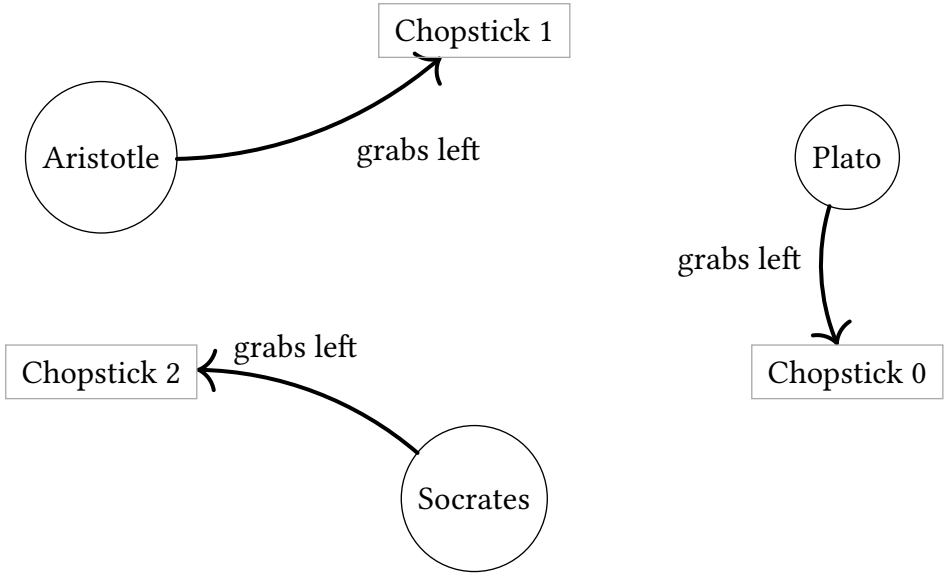


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

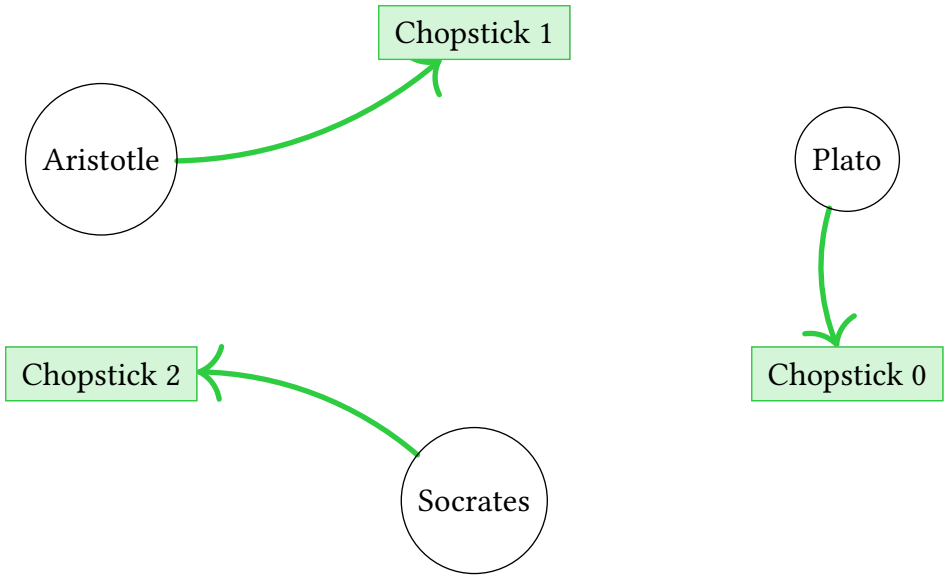


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

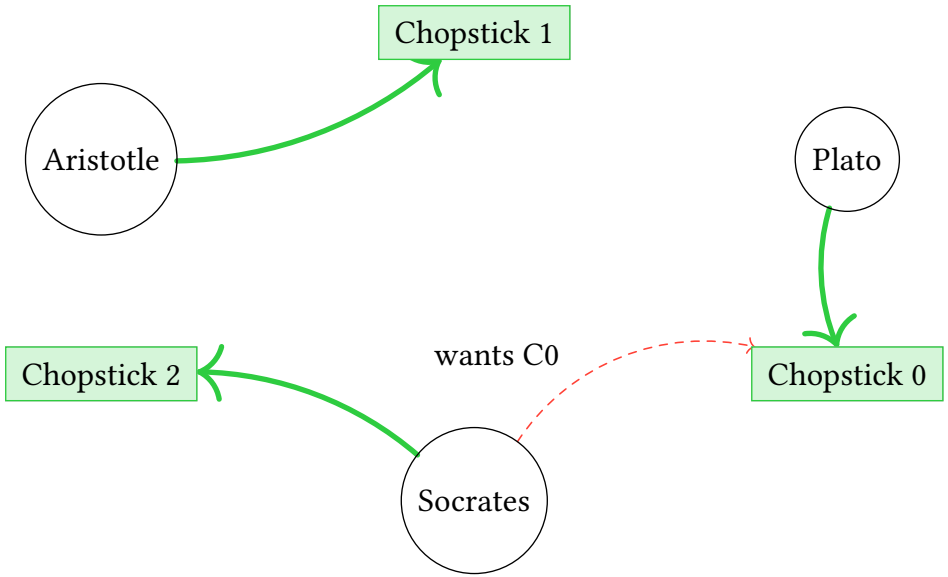


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

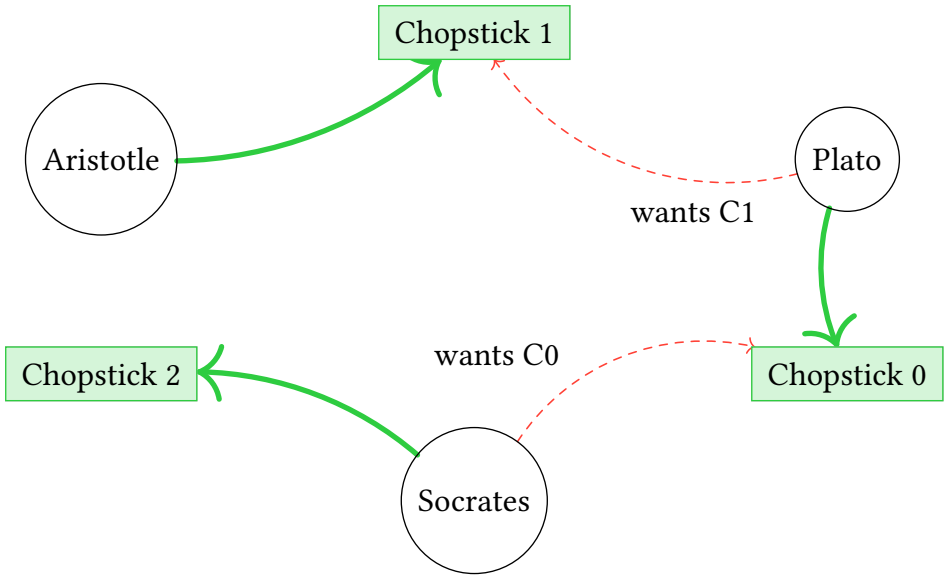


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

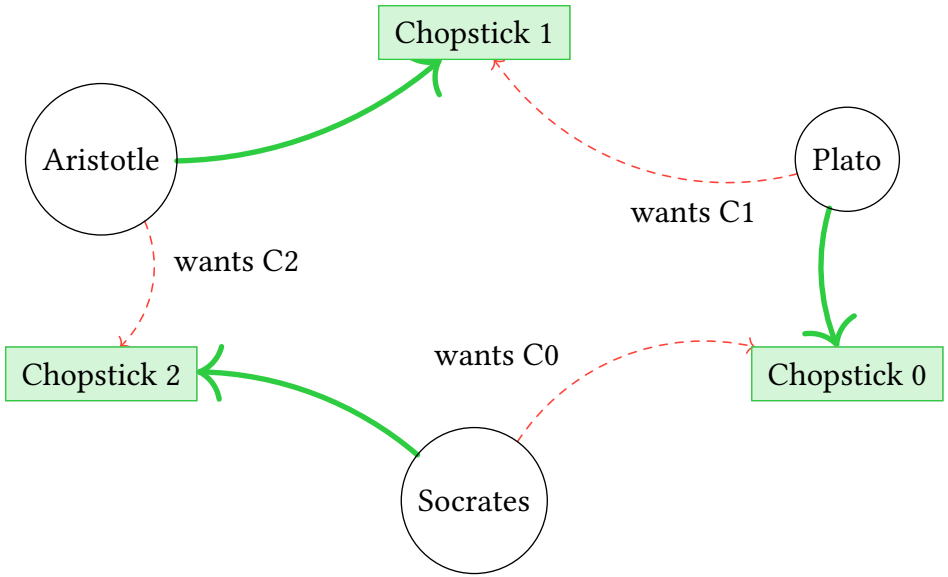


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

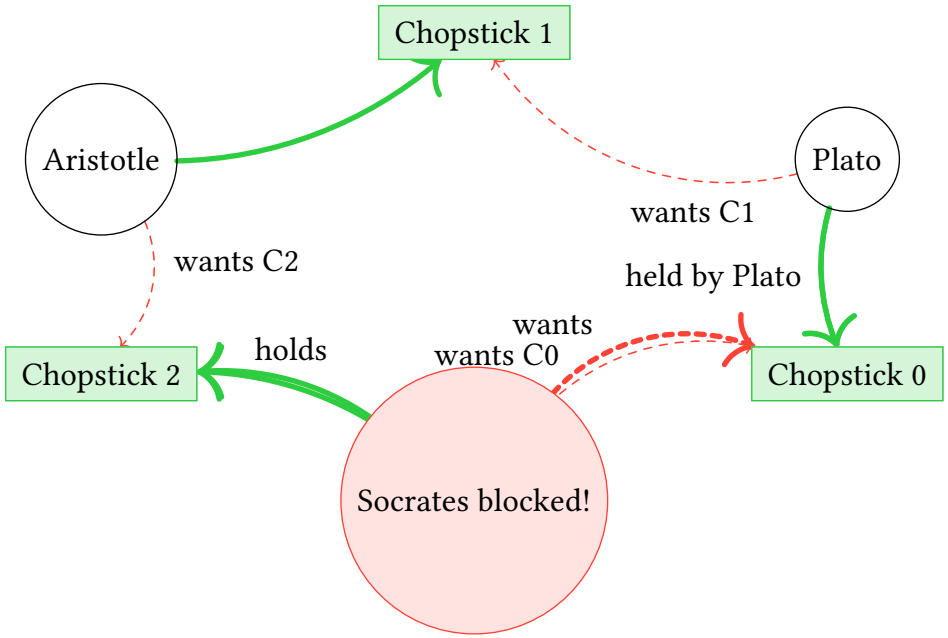


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly

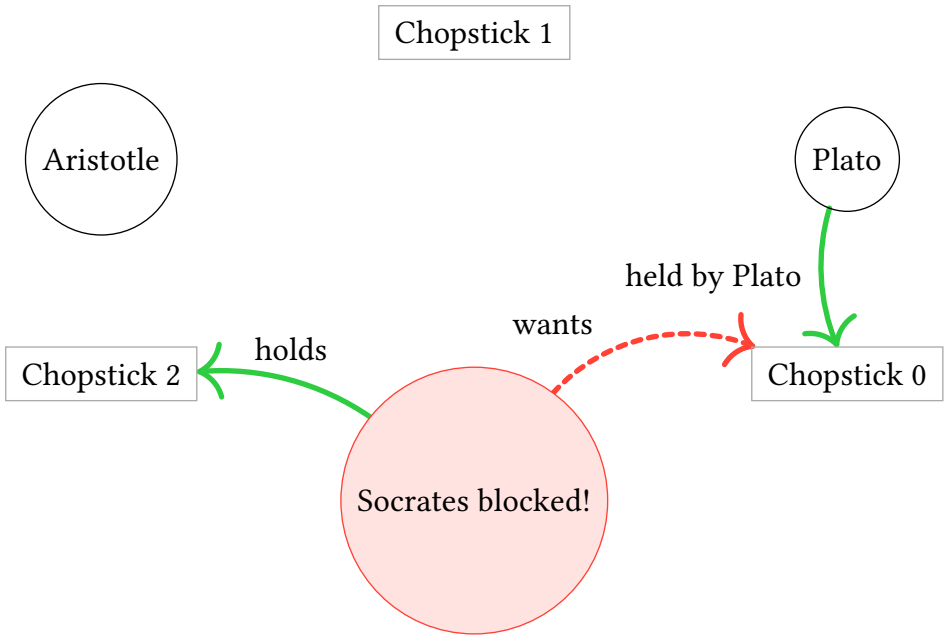


8.1. Context

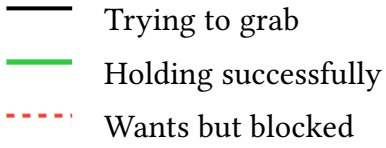
Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

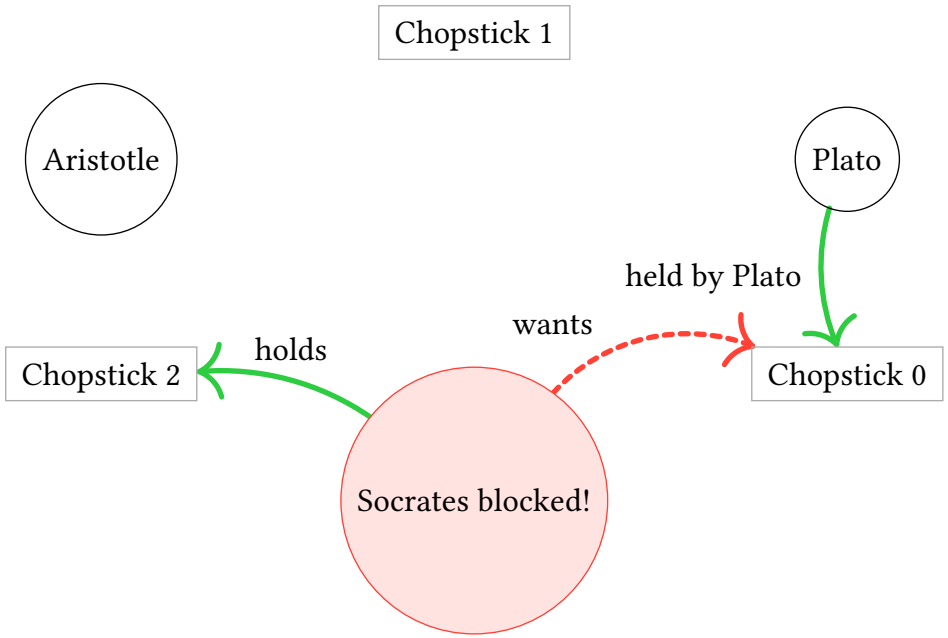


8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- Wants but blocked

Socrates' perspective:

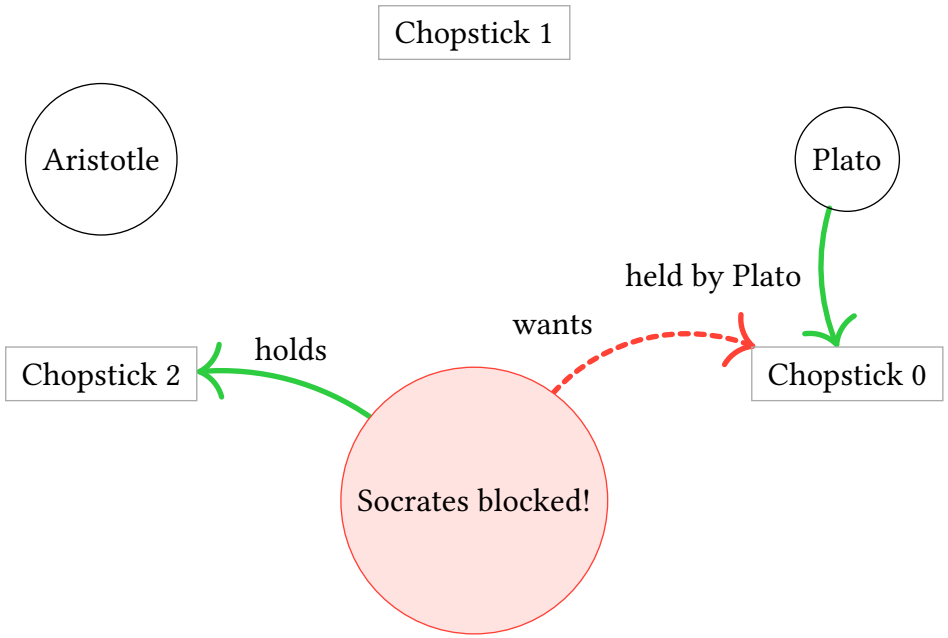
- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

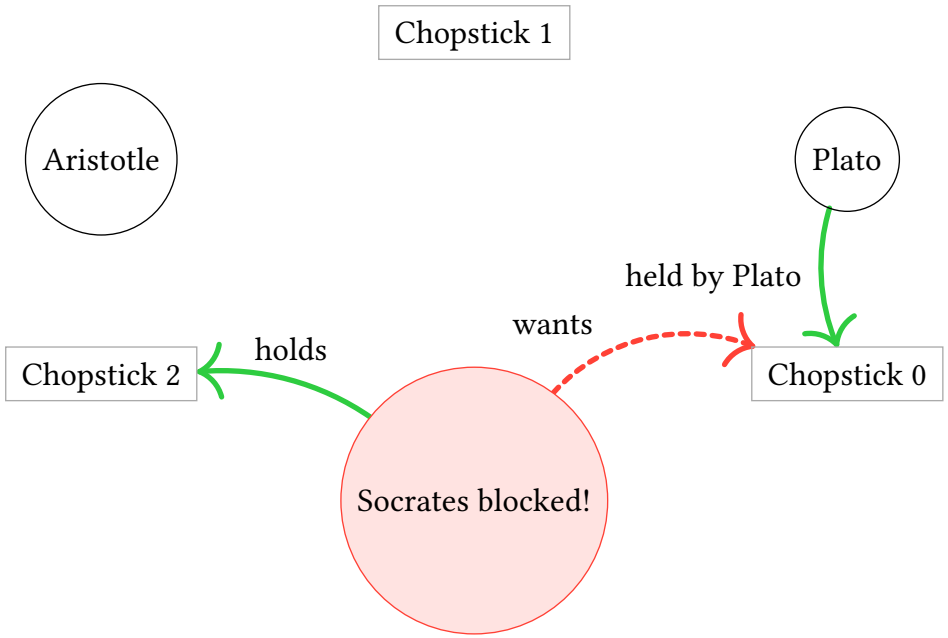
Same for Plato and Aristotle → circular wait → **Deadlock!**

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

Same for Plato and Aristotle ⟶ circular wait ⟶ **Deadlock!**

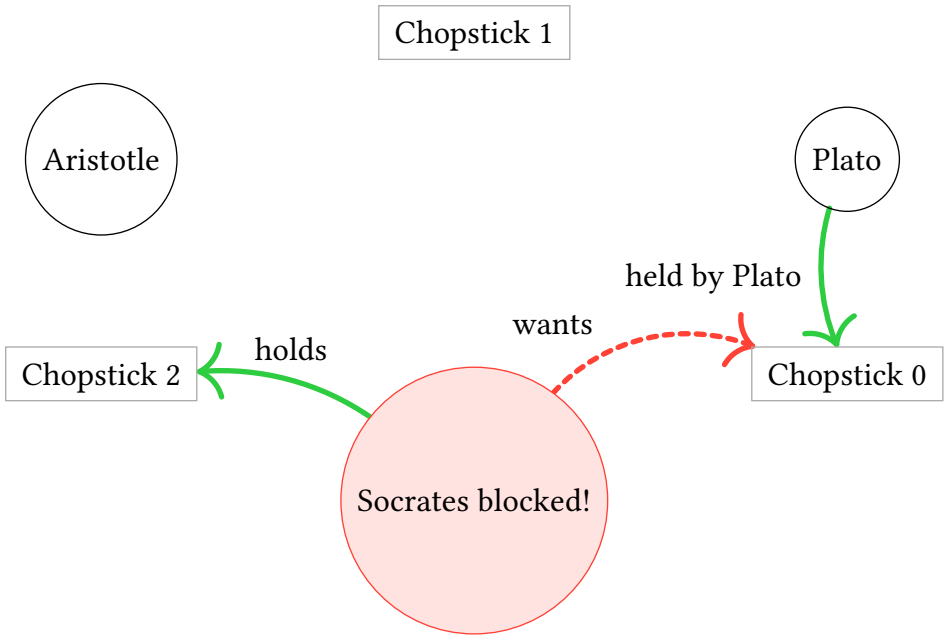
Which Coffman condition should we break?

8.1. Context

Simplified: 3 philosophers sitting around a table, 3 chopsticks between them.

Rules:

- Each philosopher needs 2 chopsticks (left and right) to eat
- Eating takes time (must hold both chopsticks during eating)
- All philosophers want to eat repeatedly



Legend:

- Trying to grab
- Holding successfully
- Wants but blocked

Socrates' perspective:

- Holds C2 (green)
- Wants C0 (red dashed)
- But C0 is held by Plato!

Same for Plato and Aristotle ⟶ circular wait ⟶ **Deadlock!**

Which Coffman condition should we break?

Circular wait: change lock order for one philosopher.

8.2. Assignment

Solve the “Dining Philosophers” problem with async tasks instead of threads.

- Revisit the standard thread-based version in `session6/examples/philosophers.rs`
- Complete exercise code in `session7/examples/philosophers.rs`.

Solutions are provided as `*-solution.rs` files.

8.3. Additional bonus

Do we still need the asymmetry (the last philosopher switching hands)?

8.3. Additional bonus

Do we still need the asymmetry (the last philosopher switching hands)?

Yes, because all philosophers are still working in parallel in separate tasks, so we may have a deadlock.

How to solve this problem with only a single async task? Do we still have deadlocks

8.3. Additional bonus

Do we still need the asymmetry (the last philosopher switching hands)?

Yes, because all philosophers are still working in parallel in separate tasks, so we may have a deadlock.

How to solve this problem with only a single async task? Do we still have deadlocks

No, since philosophers access sequentially.